

Projeto da Disciplina

1. Objetivo

Construir, de ponta a ponta, uma aplicação orientada a dados (data-driven app): a equipe escolhe uma fonte de dados real, executa um pipeline ETL que carrega esses dados no MongoDB, modela a informação como documentos (decidindo embedding x referência pelo padrão de acesso), implementa consultas que respondem a perguntas concretas e expõe o resultado em uma aplicação de visualização (dashboard web/mobile, API consultável ou similar).

O projeto consolida, em um único entregável, o conteúdo do semestre: pipeline data-driven e metadados, coleta de dados na web/APIs com cuidado de LGPD, modelos de persistência e a pergunta "quando NoSQL se justifica" e modelagem, consultas e índices no MongoDB.

2. Escopo (definição do problema)

Apresente, em até 1 página:

- a. Problema de dados: qual pergunta/necessidade a aplicação atende e para quem.
- b. Solução proposta: visão geral do pipeline ETL e da aplicação final.
- Por que documentos/MongoDB: em 3–5 linhas, justifique a escolha do modelo de documentos para este domínio ("qual problema isso resolve melhor que um PostgreSQL?"). Não precisa ser uma defesa absoluta, basta ser consciente.

3. Entendimento da Fonte de Dados

- a. Fonte: origem (API, open data, web scraping, arquivo, etc.), forma de acesso e licença/uso.
- b. Aspectos legais/éticos: se houver dado pessoal, registre o tratamento à luz da LGPD (base legal, anonimização/minimização). Se não houver, declare explicitamente.
- c. Metadados descritivos: volume, formato, frequência de atualização, idioma, fonte original.
- d. Dicionário de dados (metadados estruturais) — tabela campo | tipo | descrição | exemplo.

4. Requisitos de dados (perguntas que a aplicação responde)

Liste de 5 a 8 perguntas que os dados devem responder (ex.: "quais os 10 X mais Y por período?", "como Z se distribui por categoria?"). Cada pergunta vira, mais adiante, uma consulta (Seção 7) e, de preferência, um elemento visual (Seção 8). É este conjunto que guia toda a modelagem.

5. Modelo conceitual (UML)

Represente o modelo conceitual em **diagrama de classes UML**. Ele norteia o mapeamento para o MongoDB.

Obrigatório:

- Classes e propriedades com pelo menos um exemplo de: propriedade simples, multivalorada, composta e opcional.
- Pelo menos um relacionamento de agregação/composição.

Opcional (se fizer sentido no domínio): generalização/especialização e demais associações (1:1, 1:N, N:N).

6. Projeto do banco orientado a documentos (mapeamento)

Antes de carregar, documente o alinhamento entre o modelo conceitual, os dados coletados e as coleções no MongoDB.

Quadro A — Correspondência conceito ↔ dado coletado

Classe / Propriedade (modelo)	Metadado na fonte	Tipo geral
Livro.titulo	"Nome da obra"	String
Livro.palavrasChave	"tags"	Array
...	...	...

Quadro B — Mapeamento para o MongoDB

Elemento (modelo)	Tipo no MC	Implementação no MongoDB	Observação
Livro	Classe	Coleção	Coleção principal
titulo	Atributo simples	Campo simples	Obrigatório
tags	Multivalorado	Array	Obrigatório
endereco	Composto	Documento embutido	-
Livro → Nota	Composição	Array embutido	Lido junto com o livro
Livro → Editora	Associação	Referência (ObjectId + \$lookup)	Editora consultada à parte

Para cada relacionamento, decida **embedding** × **referência** e justifique pelo padrão de acesso ("dados lidos juntos ficam juntos"), considerando cardinalidade, limite de 16 MB do BSON, frequência de leitura conjunta e crescimento do relacionamento (regras de Coupal, 2019). Decisões arbitrárias, sem justificativa, não pontuam.

7. Pipeline ETL e carga no MongoDB

Implemente o pipeline (sugestão: organização **Medallion**: *bronze* bruto → *silver* limpo → *gold* pronto para consumo). Não basta um *import* direto da fonte: é preciso **transformar** para o modelo de documentos.

- **a. Extração** — coleta da fonte (API/scraping/arquivo), preservando o dado bruto.
- **b. Transformação / preparação** — limpeza, padronização, *type casting*, deduplicação, construção dos documentos (embutidos, arrays e referências) conforme o mapeamento da Seção 6.
- **c. Carga** — inserção no MongoDB. As coleções devem exercitar, ao final: **campos opcionais, documentos embutidos/aninhados, arrays de objetos e arrays simples**, com relacionamentos por **referência ou aninhamento** conforme decidido.
 - (*Opcional*) validação de esquema com \$jsonSchema (validator).
 - Crie os **índices** que as consultas exigem (ver Seção 7.e).
- **d. Consultas** — implemente consultas que respondam aos requisitos da Seção 4, cobrindo **no mínimo**:
 - Filtros e **projeções** (find);
 - Acesso a **arrays** e a **estruturas embutidas** (*dot notation*, \$elemMatch);
 - **Aggregation pipeline** com múltiplos estágios/operadores (\$match, \$group, \$sort, \$project, ...);
 - **\$lookup** (se houver relacionamento por referência);
 - (*se aplicável*) **\$graphLookup** para relacionamento hierárquico/recursivo;
 - Consultas específicas dos requisitos da aplicação.
- **e. Índices e desempenho** — crie ao menos **um índice** sobre um campo consultado e mostre, com explain("executionStats"), o ganho (COLLSCAN → IXSCAN). Se houver filtro + ordenação, aplique um índice composto seguindo a regra **ESR** (*Equality, Sort, Range*).
- **f. Schema Design Patterns** (*diferencial*) — aplique e justifique pelo menos **um** padrão da aula 08 (*Extended Reference, Subset, Computed, Bucket, Outlier* ou *Schema Versioning*) onde fizer sentido.

8. Aplicação de visualização (consumo dos dados)

Construa a aplicação que **consome as consultas** da Seção 7 e as apresenta ao usuário — dashboard, relatório interativo, API consultável ou interface web/mobile. Requisitos:

- Usar efetivamente as consultas definidas (find, aggregate, \$lookup...), e não dados pré-computados à mão;

- Oferecer **interatividade** (filtros, seleção de período/categoria) em pelo menos parte das visualizações;
- Cada pergunta da Seção 4 deve ser respondida por um elemento da aplicação.

A aplicação é **demonstrada na apresentação final**.

Sugestão de atalho para o dashboard: **MongoDB Charts**

(<https://www.mongodb.com/products/charts>). A tecnologia é livre, desde que evidencie o uso das consultas e das características dos dados no MongoDB.

9. Entregáveis

1. **Repositório Git** contendo:
 - a. README.md: documento do projeto (Seções 2–6) + instruções para executar pipeline e aplicação;
 - b. Código do **pipeline ETL** (extração → transformação → carga) reproduzível;
 - c. **Scripts das consultas** (.js/mongosh ou no código) com a query e um exemplo de resultado;
 - d. Diagrama UML (Seção 5) e os quadros de mapeamento (Seção 6);
 - e. Evidências de explain() (Seção 7.e).
2. **Aplicação de visualização** executável (ou *deploy*/instruções de execução local).
3. **Apresentação final** com demonstração ao vivo.
4. **Registro de software**: após concluído, registrar a aplicação conforme as diretrizes da **Diretoria de Inovação do IFPB** (formulário/processo alinhado com o professor).

10. Critérios de Avaliação

Critério	Peso
Pipeline ETL — coleta, transformação real e carga reproduzível	20%
Modelagem de documentos — UML, mapeamento e decisões <i>embed/ref</i> justificadas pelo padrão de acesso	20%
Consultas — cobertura mínima (filtros, projeção, arrays/embutidos, <i>aggregation</i> , <i>\$lookup</i>) e correção	20%
Aplicação de visualização — uso efetivo das consultas, interatividade, resposta aos requisitos	20%
Índices/desempenho + Schema Patterns — explain() e justificativa	10%
Clareza e reprodutibilidade — documentação, organização, demo	10%

Zera o item correspondente: *import* direto sem transformação; decisões de modelagem sem justificativa; uso de *\$lookup/\$graphLookup* "porque sim"; entrega que não roda sem modificação.

11. Cronograma sugerido (checkpoints)

Marco	Entrega parcial	Data de Entrega
C1	Escopo + fonte escolhida + dicionário de dados (Seções 2–4)	08/06/2026
C2	Modelo conceitual UML + mapeamento para MongoDB (Seções 5–6)	15/06/2026
C3	Pipeline ETL carregando no MongoDB + consultas (Seção 7)	22/06/2026
C4 (final)	Aplicação de visualização + apresentação + registro de software (Seções 8–9)	06/07/2026